

Ocar — Load & Spike Test Plan

Pre-launch capacity verification, run against staging

Version 1.0 | September 2026

This is the test plan for the load test referenced in the production scale-readiness report (step 13 of that document's build checklist). It runs against staging only, never production. This document covers what gets tested, the tool and scripts involved, the environment it runs in, and the sequence it's run in.

1. Why this test, and what it's checking

The architecture was sized on paper for 5,000–6,000 concurrent users and 1,000–2,000 rides/day. That sizing is an engineering estimate, not a guarantee. This test replaces the estimate with a measured result: real concurrent connections, real booking traffic, real GPS ping volume, against the real staging infrastructure, while we watch the exact components most likely to be the actual bottleneck (Redis and the database connection pool — not app CPU, per the original sizing analysis).

2. Tool

k6 (Grafana Labs), run from the command line. Chosen over JMeter for one practical reason at our target scale: k6 is goroutine-based (~100KB per virtual user) versus JMeter's thread-based model (~1MB per virtual user) — Grafana's own benchmarks show a single k6 instance comfortably sustains 30,000–40,000 virtual users, well past what one JMeter instance can hold. We don't need distributed load generation for 6,000 users; one k6 instance run from a temporary EC2 host in the same AWS region (`ap-south-1`) is sufficient and keeps network distance out of the results as a confounding factor.

3. The test suite

File	Type	What it checks
<code>smoke.js</code>	Smoke	1 virtual user, every code path once. Run privately before any live run — catches config or auth mistakes before they show up mid-test.
<code>main.js</code>	Load	The primary test. Ramps to target concurrency and holds, simulating sustained real-world traffic.
<code>spike.js</code>	Spike	Instant 0→N surge in 10 seconds, held, then dropped. Measures how fast the system absorbs a sudden traffic jump and whether it recovers cleanly afterward.
<code>accept-race.js</code>	Concurrency correctness	Fires multiple drivers at the same ride simultaneously to confirm exactly one wins the assignment and the rest get a clean rejection, not a double-booking.
<code>reconcile.js</code>		

	Data correctness (post-run)	Read-only DB check after a run — stuck rides, missing status history, ledger mismatches, orphaned sessions. Confirms the data behind the 200 status codes is actually correct.
<code>query-regression.js</code>	Query performance (post-run)	Compares query response times before/after seeding a large historical dataset, to catch a slow query before it reaches production volume.

4. What's being simulated

Four scenarios run concurrently inside `main.js` — this overlap (bookings happening while thousands of sockets are already open) is deliberate, since that's how real traffic behaves.

Scenario	Transport	Represents	Target scale (final ramp step)
<code>booking_flow</code>	HTTP	Riders booking rides	20 bookings/min in bursts (report's 1,000–2,000 rides/day is a fraction of this — this rate deliberately over-tests peak-minute bursts, not the 24h average)
<code>live_tracking</code>	WebSocket	Active drivers pinging GPS location every 3–5s	400 concurrent drivers
<code>rider_watch</code>	WebSocket	Riders with the app open, mostly idle	6,000 concurrent connections — this is what actually produces the "5–6k concurrent users" figure, not booking volume
<code>admin_dashboard</code>	HTTP	An ops admin watching the live dashboard while the above runs	30 polls/min against the real admin read endpoints

Driver/rider WebSocket connections are not paired to real ride rooms — they prove the server, Redis pub/sub, and the ALB can sustain the connection count and ping rate. Booking correctness under load is covered separately by `booking_flow` and `accept-race.js`.

Spike test parameters (separate run, after a clean load test)

Parameter	Value
Booking surge	0 → 300 bookings/min in 10 seconds, held 2 minutes, dropped to a 10/min baseline for a 3-minute recovery window
Rider connection storm	0 → 2,000 concurrent socket connections in 10 seconds, held 2 minutes, then closed

5. Endpoints and connections under test

Endpoint / event	Exercised by
<code>POST /api/v1/pricing/estimate</code>	<code>booking_flow</code> , <code>booking_spike</code>
	<code>booking_flow</code> , <code>booking_spike</code>

```
POST /api/v1/rides , GET /api/v1/rides/:id ,  
POST /api/v1/rides/:id/cancel
```

```
POST /api/v1/rides/:id/accept
```

 accept-race.js only

```
POST /api/v1/rides/sessions/online & /  
offline
```

 live_tracking

```
Socket.io location:update event
```

 live_tracking

```
Socket.io connection + idle hold
```

 rider_watch, rider_spike

```
GET /api/v1/admin/rides/stats , /admin/  
rides , /admin/sessions/active , /admin/  
drivers/:id/rides
```

 admin_dashboard

6. Seed data

Test data is seeded in two separate, purpose-built passes before any traffic runs — a large historical dataset sized for query-performance testing, and a smaller live pool sized for the connection-load ramp. They serve different questions and are not the same data.

Historical ride data — for query-performance testing

What	Detail
Volume	1,000,000 rides, each with a matching fare snapshot. The full lifecycle fan-out from that: ~720,000 payments, ~180,000 cancellations, ~4.28M ride-status-history rows, ~1.1M ratings — verified counts from an actual seeding run, not an estimate.
Spread	Dated across the trailing 12 months, so query performance is tested against realistic time-range distribution, not a single-day dump.
Outcome mix	72% completed, 18% cancelled, 5% no-drivers-available, the remainder split across in-flight states (accepted, driver arrived, in progress, scheduled) — a realistic distribution, not all-success data.
Ride type mix	70% one-way, 20% round-trip, 10% rental.
Riders	50,000 synthetic accounts, tagged with a reserved phone-number range so they're never confused with real users and can be cleanly removed afterward.
Drivers	Reuses real, already-onboarded active drivers only — nothing fabricated here — so the seeded rides exercise the same driver-matching and category logic real traffic would.

Live traffic pool — for the connection-load ramp (\$9)

Separate from the above. The ramp's 400-concurrent-driver target is a pure infrastructure-capacity question (can the load balancer, autoscaling group, and Socket.io connections hold that many simultaneous sessions), so this pool is sized differently and doesn't need to be historical:

What	Detail
------	--------

Drivers	Staging's existing real active drivers, topped up with synthetic driver accounts to reach the 400 target. Each gets a vehicle and a funded wallet, tagged with its own separate reserved phone range so it's never mixed up with real drivers or with the synthetic riders above.
Same-day step	A short, separate step grants every active driver same-day verification, a real precondition for going online. This expires daily and is re-run right before each live session, not seeded once up front.

Every synthetic row across both passes carries its own dedicated, reserved identifier range and is removed by a full cleanup pass once testing is complete — nothing synthetic is left behind in staging.

7. Edge cases and failure scenarios covered

Beyond steady-state capacity, the suite deliberately targets the specific ways a system like this actually breaks under load, not just how fast it responds when everything goes right:

Edge case	How it's tested
Two drivers racing to accept the same ride	<code>accept-race.js</code> fires multiple drivers' accept calls at the same ride truly concurrently. Exactly one must win; the rest must get a clean rejection, never a crash or a double-assignment.
A driver accepting a ride they were never offered	Each race includes one driver deliberately never brought online and never offered the ride, to confirm the accept endpoint checks eligibility and not just the ride's status.
An instant traffic surge, not a gradual one	<code>spike.js</code> jumps from 0 to peak bookings and connections in 10 seconds instead of a multi-minute ramp — tests reaction time, not just sustained capacity.
Recovery after the surge passes	<code>spike.js</code> holds a reduced baseline for 3 minutes after the surge drops, to confirm latency actually returns to normal instead of staying degraded.
Admin reads competing with write load	<code>main.js</code> 's <code>admin_dashboard</code> scenario polls the real dashboard endpoints concurrently with the booking and tracking load, rather than testing reads and writes in isolation.
Slow leaks over long-running connections	A separate soak run holds moderate (not peak) load for 3–4 hours, watching for metrics that drift upward over time instead of staying flat.
Data left inconsistent by a failed or interrupted request	<code>reconcile.js</code> checks, after every run: rides stuck in a non-terminal status, missing ride-status-history rows, payment-ledger mismatches, driver sessions stuck "on trip" with no active ride, and orphaned GPS rows.
Query performance at real data volume, not an empty table	<code>query-regression.js</code> compares the same critical queries' response times before and after the 1,000,000-row seed from §6, flagging any that regress past a set tolerance.
A production-style deploy happening while traffic is live	One of the ramp steps in §9 includes triggering an actual blue/green deploy mid-run, not just idle staging. Confirms the listener cutover doesn't drop in-flight requests or open connections, and that no new connections land on the outgoing color once the flip happens.

8. Environment and what we're watching

Environment: staging — infrastructure-identical to production (same instance types, same ASG scaling policy, same ElastiCache tier), on its own Terraform state and its own non-production credentials. Never run against production.

Monitored live, in Grafana, opened before the test starts:

Metric	Why it's the one to watch
Redis: ops/sec, connected clients	Flagged as the most likely actual bottleneck — pub/sub fan-out across every driver location broadcast
Database connection pool (pooled connections vs. ceiling)	Second most likely bottleneck, per the same analysis
EC2 CPU credit balance (burstable instance type)	Confirms whether the current instance type holds up under sustained load or needs to move to a fixed-performance type
ALB target group: 5xx rate, p95 latency	Mirrors the autoscaling group's own scaling trigger — if this breaches, that's the same signal that would trigger a scale-out in production
Per-container CPU/memory (cAdvisor)	Confirms container resource limits are sized correctly, not guessed

9. Run strategy — staged, not a single jump to target load

We do not go straight to 6,000 connections. Each step below is run and confirmed clean before advancing to the next, ideally on separate days so each step gets a clear read.

Step	Drivers	Riders	Booking rate	What it confirms
1	20	200	5/min	Everything works end to end; no configuration mistakes
2	100	1,000	10/min	Redis/DB metrics move but stay flat, no early warning signs
3	250	3,000	15/min	Confirms whether autoscaling triggers as designed
4 — target	400	6,000	20/min	The number the architecture was sized for

After step 4 passes: a spike test (§4), then a longer soak run (moderate load held for 3–4 hours) to catch slow leaks — connection leaks, memory growth — that a 40-minute run can't surface.

During one of the mid-ramp steps (step 2 or 3, once load is steady but before the target run), a real blue/green deploy is triggered against staging while that load is live — see §7. This is a deliberate part of the plan, not an accident of bad timing.

10. Pass/fail thresholds

Threshold	Load test	Spike test
p95 response time	< 500ms	< 1,500ms
HTTP error rate	< 1%	< 5%
Socket connection failures	< 50	< 100

Spike thresholds are deliberately looser — a brief latency hit during a sudden surge is expected. What matters is that it doesn't error out and that it recovers once the surge passes.

11. How issues that come up during a run are handled

Something will surface during this test — that's the point of running it, and we'll say so up front rather than treating any single failure as a crisis. Each issue gets classified on the spot into one of three buckets, and the run continues unless something is actively cascading (e.g. the database running out of connections):

1. **Test artifact** — something about the test itself (an expired token, a staging-only data gap), not a real system issue.
2. **Real but non-blocking** — a genuine bug, logged as a follow-up, doesn't affect the sign-off decision.
3. **Real capacity issue** — the thing this test exists to catch. Each of these maps to a known mitigation (burst mode → larger instance type; Redis tier bump) already identified in the original sizing report.

Deliverable after the run: a short write-up comparing what actually strained first against what the report predicted, cross-referenced against the Grafana window from the run. That comparison is the real output of this exercise, not a simple pass/fail.